



成都大学

CHENGDU UNIVERSITY

本科毕业设计（论文）开题报告

题 目 基于 JavaWeb 数据探针管理系统的设计与实现

学 院 计算机学院

专 业 计算机科学与技术

学生姓名 李 鑫

学 号 201910914113 年级 2022 级

指导教师 易发胜 职称 副教授

2025 年 11 月 20 日

题目	基于 JavaWeb 数据探针管理系统的设计与实现	
课题类型	理论性课题（ ）	实践性课题（ ☒ ）
课题来源	教师科研课题（ ）	教师拟定课题（ ☒ ）
	学生自拟课题（ ）	师生联合命题（ ）

一、选题背景、目的及意义

1.选题背景

在数字化转型的浪潮中，数据已成为驱动业务创新的核心要素。然而，在实际的数据生产环境中，数据源往往具有多源异构（物联网设备、业务数据库、非结构化文件等）、分布广泛且网络环境复杂的特点。传统的“脚本式”采集方式存在极大的局限性：采集程序通常是“一次性”的，缺乏状态感知，一旦运行失败难以察觉；且面对网络波动或数据量激增时，无法动态调整策略，容易造成数据丢失或服务器宕机。

在此背景下，“数据探针”（Data Probe）的概念应运而生。本课题所定义的探针，并非简单的采集脚本，而是一种部署在数据源侧（Edge Side）的智能化守护程序（Daemon Process）。它具备三大核心能力：

- (1) 连接管理：能够长期驻留内存，稳定维持与数据库、物联网设备或文件系统的连接，并具备断线重连机制。
- (2) 变化感知：不再依赖全量扫描，而是通过监听日志（Binlog）、文件系统事件或轮询机制，自动检测数据资源的变化并主动上报，实现毫秒级的增量感知。
- (3) 按需提取：能够严格遵循管理端下发的策略指令，按要求精确提取特定字段或时段的数据，避免无效数据占用带宽。

尽管探针在技术上解决了数据采集问题，但作为一种长期运行在后台的“守护程序”（Daemon Process），其在实际大规模部署中面临着严峻的治理挑战：

- (1) 运行状态黑盒化：探针以无头模式（Headless）运行，运维人员无法直观感知其存活状态。一旦发生“假死”或“僵尸进程”，往往直到业务端数据断流才被发现，故障响应滞后。
- (2) 配置版本漂移：面对成百上千的异构节点，依靠人工 SSH 登录修改采集规则极易出错，导致不同节点的配置版本不一致（Configuration Drift），污染采集数据。
- (3) 资源竞争隐患：缺乏统一管控的探针极易在数据洪峰时抢占宿主机的 CPU 和内

存，成为“吵闹的邻居（Noisy Neighbor）”，威胁核心业务的稳定性。

然而，随着探针数量的增加，如何高效管理这些分散在各个局域网角落的“神经末梢”，如何实现探针的自动发现、状态监控以及自适应的策略调度，成为亟待解决的技术难题。

2.选题目的

本选题旨在设计并实现一套“自适应数据探针管理系统”。该系统不直接处理庞大的业务数据流，而是作为控制中枢，专注于解决探针的“生存与调度”问题。目的在于构建一套标准化的管理框架，通过 Web 界面实现对局域网内数据探针的自动发现、远程配置、生命周期管理以及性能监控。同时，引入自适应算法，使探针能够根据宿主机的负载情况（CPU/内存）自动调整采集频率，实现智能化的运维管理。

3.选题意义

- (1) 构建数据生态的“神经系统”：本系统明确了数据源、探针、管理系统与普查平台之间的架构关系。探针作为执行端负责采集，管理系统作为控制端负责调度，两者分离的设计不仅降低了耦合度，更为上层的数据普查和溯源系统提供了稳定可靠的数据输入保障。
- (2) 提升运维自动化水平：通过引入 UDP 广播或组播技术实现局域网自动发现功能，解决了传统模式下需人工逐一录入探针信息的痛点，极大降低了部署成本。
- (3) 保障系统稳定性与资源平衡：系统的“自适应”特性解决了采集任务与业务系统争抢资源的问题。通过实时监控与动态调优，既保证了数据采集的时效性，又确保了业务服务器的稳定运行，具有重要的工程应用价值。
- (4) 实现后台服务的“白盒化”治理：针对后台守护程序“看不见、摸不着”的特性，本系统通过 JavaWeb 可视化控制台，将原本黑盒运行的进程状态实时映射为可视化心跳数据。这使得运维模式从被动的“故障后救火”转变为主动的“预防式治理”，为大规模分布式探针的管理提供了标准化的解决方案。

二、国内外研究现状和发展趋势

1.国内外行业现状

在数据采集与集成领域，国外已有较为成熟的工具，如 Apache NiFi、Logstash 等。这些工具功能强大，支持丰富的数据源插件，在大规模数据流处理上表现优异。然而，它们通常部署较重，资源消耗大，且更多侧重于“数据管道”的构建，对于分布在边缘端、资源受限环境下的轻量级探针管理（Agent Management）支持较弱。

在国内，随着工业互联网和大数据局的发展，华为、阿里等云厂商推出了各类边缘计算节点管理服务，如 **IoT Edge** 等。这些商业产品虽然实现了云边协同，但通常封闭性较强，依赖特定的云生态，且价格昂贵，不适合中小型企业或特定内网环境下的定制化需求。目前，大多数企业在进行数据普查或资产盘点时，仍采用分散的脚本管理方式，缺乏统一的、可视化的探针全生命周期管理平台。

此外，虽然 **Flume**、**Filebeat** 等开源采集组件在数据传输性能上表现优异，但它们普遍存在“重采集、轻管理”的结构性偏差。这些工具往往缺乏统一的图形化配置界面（**GUI**），在大规模集群管控、配置版本管理和可视化监控方面能力薄弱，无法满足企业对后台程序“可管可控”的运维诉求。

2. 现有技术的发展

随着云原生和微服务技术的发展，**Agent**（代理/探针）技术正向着轻量化、智能化方向演进。

通信机制的进化：从传统的 **HTTP** 轮询转向基于 **WebSocket**、**MQTT** 等长连接协议，实现了服务端对探针的毫秒级实时监控与指令下发。

自适应能力的增强：传统的静态配置正被动态策略取代。现代探针程序开始集成简单的 **AI** 或规则引擎，能够根据本地系统的 **CPU** 水位、内存压力自动“降频”或“休眠”，体现了边缘智能（**Edge Intelligence**）的趋势。

自动发现技术的应用：在局域网设备管理中，**mDNS**、**SSDP** 以及基于 **UDP** 广播的服务发现协议被广泛应用，使得设备接入实现了“即插即用”，这也是本课题拟重点突破的技术点之一。

综上所述，开发一款轻量级、具备自适应能力且支持局域网自动发现的数据探针管理系统，既符合技术轻量化的发展趋势，也能精准解决当前数据治理中的实际痛点。

三、主要研究内容、拟解决的问题及预期成果

1. 研究内容

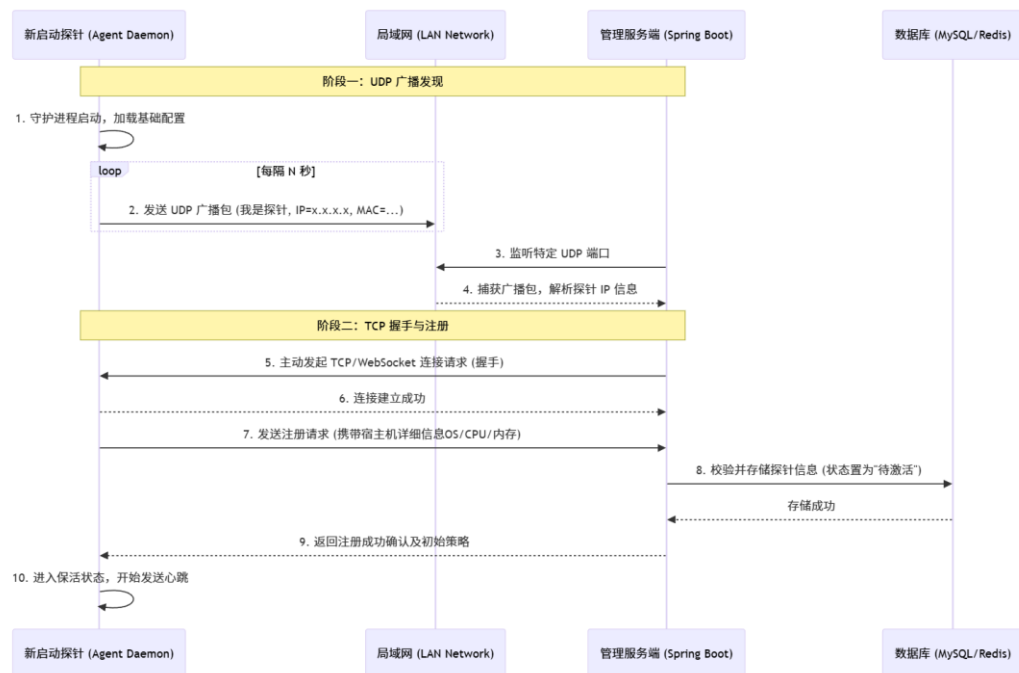
(1) 探针守护进程（**Agent**）的核心功能研发（此处为本次重点修改） 本系统需开发一个轻量级、高可用的探针 **Agent** 程序，其内部逻辑包含三个核心模块：

- ◆ 多源连接管理模块：设计统一的连接池（**Connection Pool**），支持同时管理 **JDBC**（数据库）、**MQTT**（物联网）、**I/O Stream**（文件）等多种异构数据源的连接句柄。实现连接状态的实时监测，一旦发生网络抖动或源端重启，探针需自动执行退避重连算法，确保连接不断线。
- ◆ 变化感知与主动上报模块：改变传统的“定时拉取”模式，实现基于事件驱动的主动上报。对于数据库资源，解析 **Binlog** 或 **Redo Log** 实时捕获增量变化；对于文件资源，利用 **File Watcher** 监听目录变动。一旦检测到数据更新，探针立

即将元数据（而非全量数据）推送至管理端或下游平台。

- ◆ 策略驱动的数据提取模块：在探针本地内置轻量级规则引擎（Rule Engine）。探针接收到管理端的采集指令（如“仅采集温度>30℃的数据”或“仅同步 User 表的 update 操作”）后，在本地进行数据的预处理、过滤和格式化，严格按照要求提取数据，实现“边缘清洗”，减少网络传输压力。

- (2) 局域网自动发现功能：利用 UDP 广播/多播技术，实现管理端对同一网段内新启动探针的自动侦测与握手注册，解决人工配置繁琐的问题。
- (3) 自适应配置策略算法：研发策略调度引擎，根据探针上报的宿主机性能指标（CPU/内存/磁盘 IO）及业务优先级，动态计算并下发采集频率、批次大小等参数。
- (4) 全生命周期可视化管理：基于 Web 架构开发管理控制台，实现探针的注册、激活、升级、停用及注销的全流程管理，并提供可视化的状态监控大屏。



图一 • 探针局域网自动发现与注册流程图

2.拟解决的问题：

- (1) 异构环境下的统一管理：如何屏蔽不同类型探针（物联网采集、数据库同步、文件扫描）的底层差异，设计统一的通信协议与管理接口。
- (2) 守护进程的存活判定与状态同步：如何在高并发环境下设计高效的心跳检测机制（Heartbeat），准确区分“网络抖动”与“进程僵死（Zombie Process）”。解决探针在后台静默崩溃后，管理端如何快速感知并触发告警或自愈流程，避免“失联”探针长期占用管理资源。
- (3) 资源竞争与调度平衡：如何设计合理的算法，在保证采集任务完成度的前提下，最大限度减少探针程序对业务服务器正常运行的影响。

3. 预期的成果：

- (1) 软件系统：完成“自适应数据探针管理系统”的 Web 服务端及配套的“探针 Agent”程序开发。
- (2) 技术文档：产出系统架构设计图（包含数据源、探针、管理平台关系）、API 接口文档及用户操作手册。
- (3) 毕业论文：撰写毕业设计论文，详细阐述局域网发现机制、自适应算法及系统实现过程。

四、拟采用的研究思路

1.研究方法

(1) 文献研究法：

深入调研国内外关于分布式 Agent 管理、边缘计算调度及服务发现协议的相关文献，确定通信协议选型。

(2) 原型开发法：

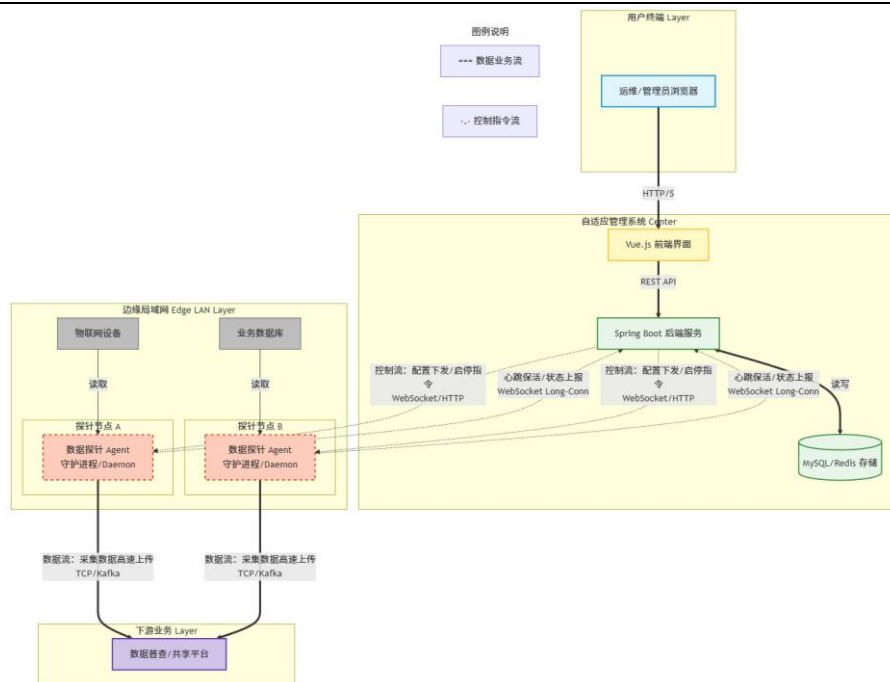
优先开发核心的“探针-服务端”通信模块，验证心跳保活与 UDP 广播发现的可行性，再逐步扩展 Web 管理功能。

2.技术路线

(1) 系统总体架构

系统采用经典的 B/S (Browser/Server) 架构。选择 Web 架构的原因在于其具有集中管控、无客户端部署、易于维护的优势，运维人员通过浏览器即可管理分布在不同物理位置的探针。

数据流向说明：架构分为三层。底层为数据源与探针；中层为管理系统；上层为业务应用。需要强调的是，探针采集的业务数据流不经过管理系统，直接推送至数据普查平台；管理系统仅负责控制流（配置、状态、指令），这种“数控分离”的设计保证了系统的高效性。



图二 • 系统总体架构与“数控分离”流向图

(2) 采用技术栈

后端：采用 Java (Spring Boot) 构建微服务，集成 Netty 或 Spring WebSocket 处理与探针的长连接通信；利用 UDP 协议实现局域网内的服务发现。

前端：采用 Vue.js + Element UI 构建单页面应用（SPA），使用 ECharts 实现探针性能数据的动态可视化展示。

探针端（Agent）：基于 Java 或 Go 编写轻量级守护程序，封装系统监控库（如 OSHI）采集本机资源数据。

数据存储：MySQL 存储元数据（探针列表、配置规则），Redis 存储实时心跳状态与监控热数据。

3. 可行性论证

- (1) 技术可行性：Spring Boot 和 Vue 是成熟的主流开发框架；UDP 广播和 WebSocket 也是标准的网络协议，技术风险可控。
- (2) 环境可行性：开发环境只需普通 PC 安装 JDK、Docker 和 IDE 即可；测试环境可通过 Docker 容器模拟多个探针节点，无需大量物理设备。
- (3) 能力基础：在校期间已修习 Java Web 开发、计算机网络及操作系统课程，对多线程编程和网络 Socket 通信有一定理论与实践基础，具备完成本课题的能力。

五、主要参考文献

- [1]. Walls C. Spring Boot 实战[M]. 丁雪丰, 译. 北京: 人民邮电出版社, 2016.
- [2]. Maurer N, Wolfthal M. Netty in Action[M]. Manning Publications, 2015.
- [3]. 黄邦伟. Vue.js 前端开发实战[M]. 北京: 清华大学出版社, 2019.
- [4]. Harold E R. Java Network Programming[M]. 4th Edition. O'Reilly Media, 2013.
- [5]. Carlson J L. Redis in Action[M]. Manning Publications, 2013.
- [6]. Wang Y, Li H, Sutcliffe R, et al. A Data-Driven Adaptive Sampling Method Based on Edge Computing[J]. Sensors, 2020, 20(8): 2174.
- [7]. Harb H, Makhoul A. Energy-efficient sensor data collection approach for industrial process monitoring[J]. IEEE Transactions on Industrial Informatics, 2018, 14(2): 661-672.
- [8]. Liu Y, Zhang H. Design of intelligent monitoring agent based on heartbeat mechanism[J]. Journal of Systems Architecture, 2021, 115: 102-110.
- [9]. Newman S. Building Microservices: Designing Fine-Grained Systems[M]. O'Reilly Media, 2015.
- [10]. Xu Z, Di L, Zhang H. Adaptive Data Collection and Offloading in IoT Systems: A Deep Reinforcement Learning Approach[J]. Remote Sensing, 2023, 15(2): 292.
- [11]. 杭州海康威视数字技术股份有限公司. 基于 UDP 广播发现局域网内设备的方法: CN112436950A[P]. 2021-03-02.
- [12]. 李明. 基于 Agent 的服务器性能监控系统的设计与实现[D]. 北京: 北京邮电大学, 2019.
- [13]. 张伟. 分布式环境下海量数据采集与传输系统的研究与实现[D]. 成都: 电子科技大学, 2020.
- [14]. 罗军舟, 吴文甲, 杨明. 边缘计算: 万物互联时代的新型计算模型[J]. 计算机科学与探索, 2019, 13(1): 1-16.
- [15]. Pressman R S. Software Engineering: A Practitioner's Approach[M]. 8th Edition. McGraw-Hill Education, 2014.

六、工作安排及进度

2025.12.01 – 2025.12.20: 文献阅读与科学调研

2025.12.20 – 2026.01.01: 完成系统架构设计

2026.01.01 – 2026.01.20: 完成后端基础框架搭建，实现探针与服务端的 WebSocket 通信及 UDP 自动发现功能。

2026.01.20 – 2026.02.15: 完成 Web 管理端界面开发及自适应配置算法的实现。

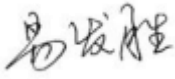
2026.02.15 – 2026.03.01: 系统联调，进行压力测试，优化心跳机制与界面交互体验。

2026.03.01 – 2026.03.10: 查漏补缺，功能完善。

2026.03.10 – 2026.04.30: 撰写毕业论文，准备答辩。

开题报告会议纪要

时 间	2025.11.25	地 点	10128	主持人	易发胜
参 会 教 师	姓 名	职 务（职 称）	姓 名	职 务（职 称）	
	易发胜	副教授			
	郑加林	讲师			
	张健伟	副教授			
会 议 记 录 摘 要	问：本课题的研究背景及拟解决的核心痛点是什么？				
	答：当前企业数据采集面临静态探针无法适应云原生环境、资源利用率波动大、故障发现不及时三大挑战。本系统旨在通过智能调度和动态配置，显著提升数据采集的实时性，优化资源利用率并实现探针故障自动恢复。				
	问：系统的总体架构及核心技术创新点有哪些？				
	答：系统采用“三横三纵”分层架构，包含感知层、决策层和执行层。技术创新点包括：基于改进 PPO 算法的动态探针编排、基于 JSON Schema 与 gRPC 的自适应配置热更新，以及采用改进 Raft 协议实现的分布式故障自愈机制。				
	问：系统的功能模块设计与数据存储方案是如何规划的？				
	答：系统设计了探针管理、资源调度、配置中心等六大核心模块。存储方案采用 PostgreSQL 存储结构化数据，InfluxDB 存储时序数据，以满足监控指标的高效读写需求。				
	记录人：				
	2025 年 11 月 25 日				

指导教师意见	该选题紧扣工业互联网与金融科技领域的数据采集需求，针对云原生环境下探针管理的实时性与稳定性问题提出了创新解决方案，具有较强的理论意义与应用价值。 开题报告结构完整，技术路线清晰，所提出的基于改进 PP0 的调度算法与去中心化故障自愈机制具有较高的技术含量。研究计划安排合理，预期成果明确。建议在后续开发中重点关注多源异构数据适配的兼容性测试，并进一步验证算法在高并发场景下的收敛性与性能。同意开题，请严格按照进度开展后续研究工作。 签字：  2025 年 11 月 25 日
--------	---

备注：1. 本开题报告除“开题报告会议纪要”各栏目外，其它栏目均由学生填写。

2. 填写各栏目时可根据内容另加附页。

3. 参加开题报告会议的教师不少于 3 人。